

INT315 • CLUSTER COMPUTING

UNIT VI

Spark ML Programming and PySpark

MLlib, the ML Pipeline API, Six Core Algorithms & Model Evaluation

Theory -> Comic Visuals -> Industry Context -> Commands/Code -> Self-Assessment

Six syllabus topics, in full depth, with heavy runnable PySpark code and seven original diagrams

Session 2026-27

Companion Study Material — INT315 Cluster Computing, Unit VI

Table of Contents

Unit VI at a Glance	4
1. Introduction to Machine Learning and PySpark.....	5
1.1 What Machine Learning Actually Is	5
1.2 Why Machine Learning at Big Data Scale Needs Spark	5
1.3 PySpark's Role	5
2. Techniques of Machine Learning	7
2.1 Supervised Learning.....	7
2.2 Unsupervised Learning	7
2.3 Reinforcement Learning (Briefly, for Completeness)	8
2.4 Matching Techniques to This Unit's Algorithms	8
3. Introduction to Spark MLlib	10
3.1 Two APIs Under One Name.....	10
3.2 What MLlib Actually Provides	11
3.3 Model Persistence	11
4. Key Concepts of Spark ML	13
4.1 DataFrame — the Universal Data Structure	13
4.2 Transformer	13
4.3 Estimator.....	13
4.4 Pipeline	14
4.5 Parameter	14
5. Spark ML Algorithms Using PySpark.....	16
5.1 Linear Regression	16
5.2 Logistic Regression.....	17
5.3 Decision Tree	18
5.4 K-Means Clustering.....	18
5.5 Support Vector Machine (SVM)	19
5.6 Naive Bayes.....	20
5.7 Comparing All Six Algorithms.....	20
6. Evaluation and Performance Metrics	22
6.1 The Confusion Matrix.....	22
6.2 Regression Metrics — RMSE, MAE, and R^2	23
6.3 Correlation Heat Maps — Understanding Features Before Modeling	24
Unit VI Review	27

Unit VI at a Glance

Every prior unit built toward this one. Unit III's feature-extraction section previewed the `fit()/transform()` pattern; Unit IV's DataFrames are exactly what Spark ML trains models on; Unit V's Kafka streams are exactly what a deployed model scores in real time. This unit closes the loop: Spark's machine learning library, MLlib, and its modern DataFrame-based API, `spark.ml` (exposed to Python as `pyspark.ml`), let you train, evaluate, and apply real predictive models across a distributed cluster using the same DataFrame you've been using since Unit IV.

This unit covers machine learning from first principles — what it is, how it's categorized, MLlib's own architecture and core concepts — before working through six concrete algorithms with full, runnable PySpark code: Linear Regression, Logistic Regression, Decision Tree, K-Means, SVM, and Naive Bayes. It closes with the evaluation toolkit every one of those algorithms needs: the confusion matrix, R^2 , RMSE, MAE, and correlation heat maps. Following Units III-V's lead, expect dense, heavy code throughout, plus seven new diagrams covering the ML workflow, MLlib's architecture, the Pipeline API, algorithm selection, and evaluation metrics.



From Following a Recipe to Writing Your Own

Every earlier unit taught Spark to faithfully execute instructions YOU wrote -- filter this, join that, aggregate the other thing. Machine learning asks Spark to do something categorically different: look at a pile of past examples and WRITE ITS OWN RECIPE for predicting the future, a recipe no human explicitly typed out line by line. You still write the code that sets this in motion -- which algorithm, which features, which data -- but the actual predictive logic (the trained model) is something Spark discovers from data, at scale, across your whole cluster.

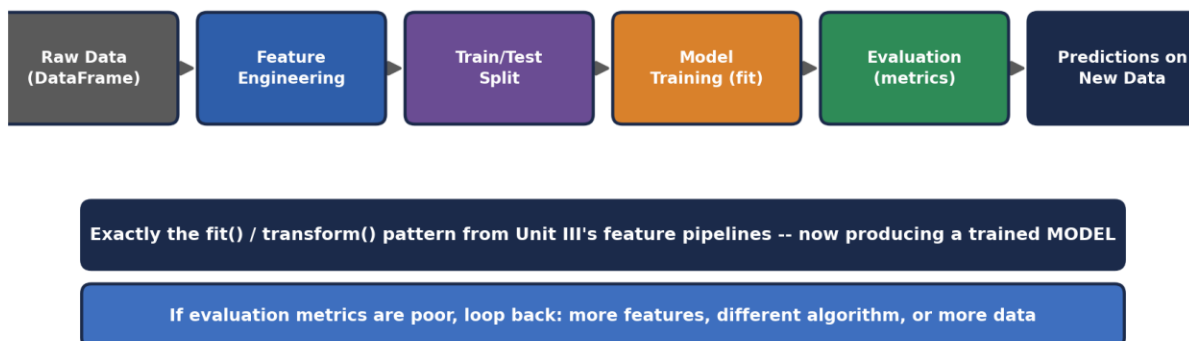
1. Introduction to Machine Learning and PySpark

This section establishes what machine learning actually is, why doing it at Big Data scale specifically requires a tool like Spark, and what PySpark's role is in that story.

1.1 What Machine Learning Actually Is

Machine learning is the practice of having a computer system learn a predictive PATTERN from data, rather than being explicitly programmed with hand-written rules for every case. Instead of a developer writing "if the transaction amount is over \$10,000 AND it's from a new device THEN flag it as fraud" (a hand-coded rule), an ML algorithm is shown thousands of past transactions ALREADY labeled fraud or not-fraud, and it learns the underlying pattern itself -- often finding subtle combinations of signals no human rule-writer would have thought to encode.

The End-to-End Machine Learning Workflow in Spark



The End-to-End Machine Learning Workflow in Spark

1.2 Why Machine Learning at Big Data Scale Needs Spark

Traditional ML tools (scikit-learn, for instance) assume the ENTIRE training dataset fits in one machine's memory. Once a training set grows into the tens or hundreds of millions of rows — the routine scale of Units III-V's material — that assumption breaks down entirely. Spark ML extends the exact same distributed, partitioned execution model from every earlier unit to model training itself: training data lives as a partitioned DataFrame across the cluster, and the training ALGORITHM itself is implemented to compute in a distributed fashion, aggregating partial results across executors exactly the way Unit IV's `groupBy + agg` does.

1.3 PySpark's Role

PySpark is the Python API for Spark, letting Python code drive the exact same distributed engine used by Scala throughout Units III-V (recall Unit I's point that Spark itself is written in Scala, with Python bridging over via Py4J). For machine learning specifically, PySpark is by far the most commonly used language in practice — Python's dominance in the broader data science and ML ecosystem (NumPy, pandas, scikit-learn) makes `pyspark.ml` a natural, familiar entry point for data scientists who need to scale existing skills up to distributed data.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("MLDemo").getOrCreate()

# Reading data exactly as in Unit IV -- ML starts from an ordinary DataFrame
df = spark.read.option("header", "true").option("inferSchema", "true") \
    .csv("hdfs:///data/customers.csv")
df.printSchema()
df.show(5)
```

Why PySpark specifically dominates applied ML on Spark

A large majority of real-world Spark ML code in industry is written in PySpark rather than Scala, precisely because most data scientists already know Python deeply and only need to learn Spark's DataFrame API on top of that existing skill — not a new programming language altogether. Companies commonly maintain Scala for performance-critical core Spark pipeline code while defaulting to PySpark specifically for the ML modeling layer, where developer familiarity and rapid iteration matter more than the small performance gap between the two languages for this kind of workload.

Teaching a New Employee vs. Writing an Employee Handbook

Traditional software development is writing an exhaustive employee handbook covering every single situation a worker might ever encounter -- painstaking, complete, and brittle the moment a genuinely new situation shows up that the handbook never anticipated. Machine learning is more like hiring a new employee and having them shadow thousands of past cases handled by your best people, letting them absorb the underlying JUDGMENT rather than memorizing a fixed rulebook -- and Spark ML is simply doing this training process across an entire WAREHOUSE of employees (executors) simultaneously, rather than training just one person alone.

Check yourself — Section 1

- Q1. In your own words, what's the fundamental difference between traditional rule-based programming and machine learning?
- Q2. Why does training data at Big Data scale specifically require a distributed tool like Spark ML rather than a single-machine library?
- Q3. Why has PySpark become the dominant language for applied Spark ML work in industry, despite Spark itself being written in Scala?

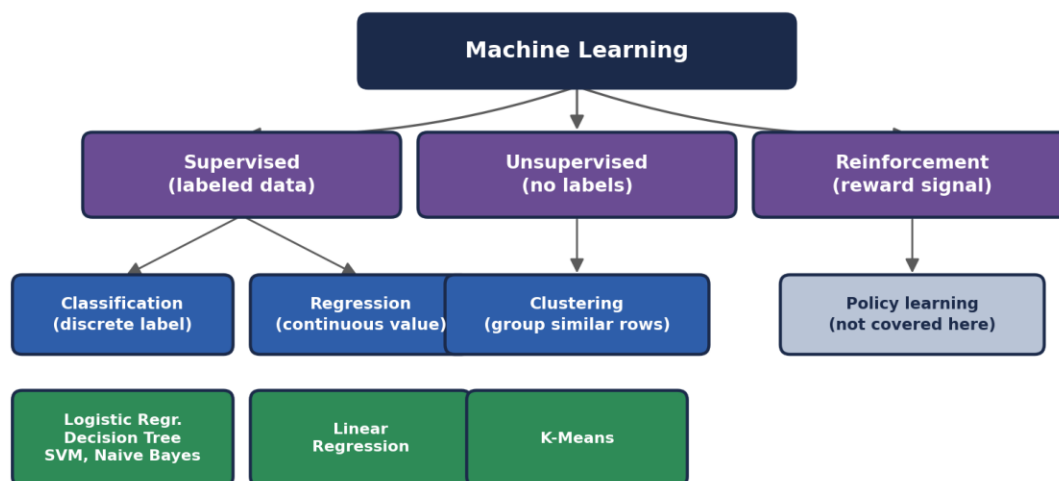
Hands-on Challenge:

Load any CSV dataset you have access to (or create a small one with at least 20 rows and 3-4 columns) into a Spark DataFrame using pyspark, print its schema, and compute basic summary statistics using `.describe().show()`.

2. Techniques of Machine Learning

Before touching any specific algorithm, this section establishes the taxonomy every ML algorithm in this unit falls into -- the vocabulary that determines which algorithm is even a candidate for a given problem.

Techniques of Machine Learning — A Taxonomy



This unit covers 5 supervised algorithms (classification + regression) and K-Means (clustering)

All trained the exact same way in Spark ML: an Estimator's `.fit()` produces a Model (Transformer)

Techniques of Machine Learning — A Taxonomy

2.1 Supervised Learning

In supervised learning, every training example comes with a known, correct LABEL -- the "answer key" the algorithm learns to reproduce. Supervised learning splits further into two shapes based on what kind of label is being predicted:

- Classification — the label is a discrete category (spam / not spam, fraud / not fraud, which of 5 product categories). Section 5 covers Logistic Regression, Decision Tree, SVM, and Naive Bayes, all classification algorithms.
- Regression — the label is a continuous numeric value (a house price, a temperature, a customer's predicted lifetime spend). Section 5 covers Linear Regression, the classic regression algorithm.

2.2 Unsupervised Learning

In unsupervised learning, training data has NO labels at all — the algorithm's job is to find structure in the data on its own. CLUSTERING, the most common unsupervised technique, groups similar rows together without ever being told what the groups "mean." Section 5 covers K-Means, the classic clustering algorithm, used constantly for customer segmentation, anomaly detection, and document grouping.

2.3 Reinforcement Learning (Briefly, for Completeness)

A third major branch, reinforcement learning, trains an agent to take actions in an environment to maximize a cumulative reward signal (think: a game-playing AI, or a robot learning to walk) — genuinely different in structure from supervised/unsupervised learning and not covered by Spark MLlib's algorithm set, but worth naming for a complete picture of the field.

2.4 Matching Techniques to This Unit's Algorithms

Technique	Algorithms Covered in Section 5
Classification (supervised)	Logistic Regression, Decision Tree, SVM, Naive Bayes
Regression (supervised)	Linear Regression
Clustering (unsupervised)	K-Means

A subtle but important naming trap

Despite its name, Logistic Regression is a CLASSIFICATION algorithm, not a regression algorithm — it predicts a discrete category (commonly binary: yes/no) by internally modeling a probability, not a continuous numeric value. This naming inconsistency, inherited from statistics, trips up a genuinely large number of ML newcomers, and is worth memorizing explicitly before Section 5.

A Teacher With an Answer Key vs. a Detective With No Leads

Supervised learning is a student studying from a stack of practice exams that ALREADY have the correct answers written on the back -- the student's whole job is learning to reproduce that answer key's logic on a fresh, unseen exam. Unsupervised learning is a detective walking into a room full of unlabeled evidence with NO answer key at all, whose job isn't to match a known answer but to notice which pieces of evidence naturally seem to belong together, forming groups nobody explicitly told them to look for.

Why this taxonomy is the first question in any real ML project

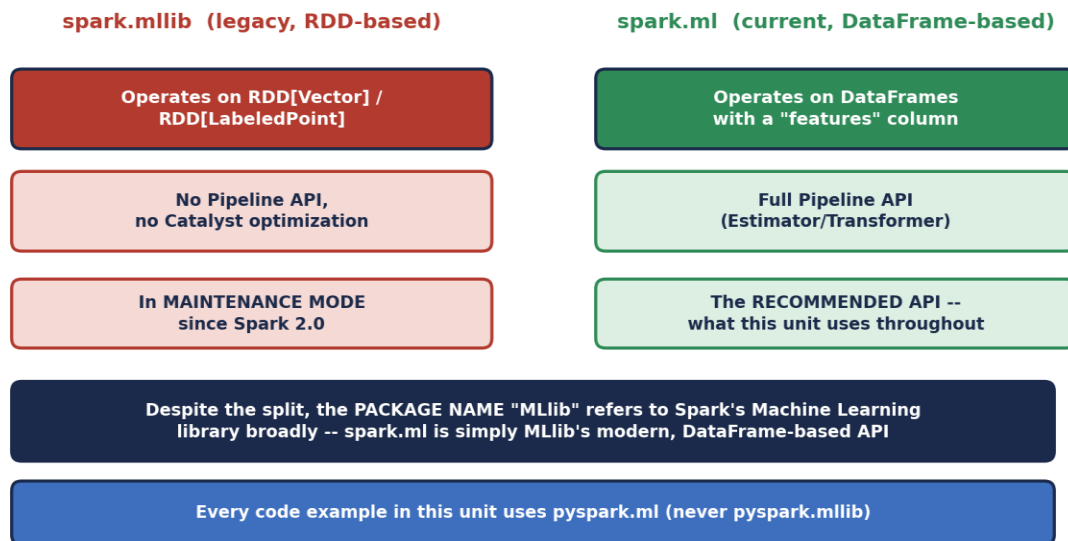
Before writing a single line of modeling code, a working data scientist's first question is always "do I have labels, and if so, are they categories or numbers?" — because that single answer determines the entire candidate algorithm list. Misidentifying a regression problem as classification (or vice versa) at the start of a project is a fundamental, project-derailing mistake that experienced practitioners are trained to catch immediately.

Technique	Algorithms Covered in Section 5
 Check yourself — Section 2 Q1. What's the key difference between classification and regression, and give one example of each that ISN'T already used in this section. Q2. Why is Logistic Regression a classification algorithm despite having "regression" in its name? Q3. If you had a dataset of customer purchase histories with NO labels at all, which technique from this section would you reach for, and why? Hands-on Challenge: For each of the following five scenarios, identify whether it's classification, regression, or clustering: (1) predicting tomorrow's stock price, (2) grouping news articles by topic with no predefined categories, (3) predicting whether an email is spam, (4) predicting a house's sale price, (5) segmenting customers into behavioral groups for marketing.	

3. Introduction to Spark MLlib

MLlib is Spark's built-in machine learning library. This section clarifies exactly what MLlib includes, and resolves a naming confusion that trips up almost everyone encountering it for the first time.

Spark MLlib — Two APIs, One Library



Spark MLlib — Two APIs, One Library

3.1 Two APIs Under One Name

"MLlib" refers to Spark's machine learning capability broadly, but it's implemented as TWO separate Python/Scala packages with genuinely different underlying data structures:

- `pyspark.mllib` — the ORIGINAL, RDD-based API, operating on `RDD[Vector]` and `RDD[LabeledPoint]` directly (recall Unit III's RDDs). It has been in MAINTENANCE MODE since Spark 2.0, receiving only critical bug fixes, no new features.
- `pyspark.ml` — the CURRENT, DataFrame-based API, operating on ordinary DataFrames with a designated "features" column (recall Unit IV's DataFrames, and Unit III's feature-extraction pipeline). This is the actively developed, recommended API, and the one this entire unit uses.

⚠ A very common source of confusion in tutorials and older material

Searching online for Spark ML examples will surface a mix of old ``pyspark.mllib`` code (RDD-based, using ``LabeledPoint``) and current ``pyspark.ml`` code (DataFrame-based, using a Pipeline). These are NOT interchangeable — mixing imports from both packages in one codebase is a common, confusing mistake. Every code example in this unit deliberately uses only ``pyspark.ml``.

3.2 What MLlib Actually Provides

Category	What's Included
Algorithms	Classification, regression, clustering, and recommendation algorithms (Section 5 covers six of these)
Featurization	Feature extraction, transformation, and selection tools — directly building on Unit III Section 10
Pipelines	Tools for constructing, evaluating, and tuning ML workflows as a single unit (Section 4)
Persistence	Saving and loading models and pipelines, so a trained model can be reused without retraining
Utilities	Linear algebra, statistics, and data-handling utilities for ML-specific needs

```
# The import you'll use throughout this unit -- always pyspark.ml, never
pyspark.mllib
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import RegressionEvaluator,
MulticlassClassificationEvaluator
```

3.3 Model Persistence

A trained model represents real, often expensive-to-reproduce computational work -- MLlib lets you save a fitted model (or an entire fitted Pipeline) to disk and reload it later without retraining, essential for any real deployment where a model is trained once and then applied repeatedly to new data over time.

```
# Saving a trained model
model.write().overwrite().save("hdfs:///models/my_logistic_regression")

# Loading it back later -- in a completely different application/session
from pyspark.ml.classification import LogisticRegressionModel
loadedModel =
LogisticRegressionModel.load("hdfs:///models/my_logistic_regression")

# The loaded model works exactly like the original -- ready for .transform()
predictions = loadedModel.transform(newData)
```



An Old Toolbox in the Attic vs. the Modern Workshop

pyspark.mllib is an old, still-functional toolbox sitting in the attic -- the tools inside still technically work, and you'll occasionally find them referenced in an old manual, but nobody's designing new tools to go in that box anymore. pyspark.ml is the modern, actively-stocked workshop downstairs, where every new tool is designed to fit together as an assembly line (the Pipeline API) rather than being used one at a time in isolation.

Why persistence matters for real deployment

Training a model on a genuinely large dataset can take hours; re-running that training every time a prediction is needed would be both wasteful and far too slow for any real-time use case (recall Unit V's fraud-detection latency requirements). Real production ML systems train a model on a schedule (nightly, weekly), save it via exactly this persistence mechanism, and load the already-trained model into a fast-serving process — often a Spark Structured Streaming job reading from Kafka, tying directly back to Unit V's material.

Check yourself — Section 3

- Q1. What's the difference between `pyspark.mllib` and `pyspark.ml`, and which one should new code always use?
- Q2. Why is it a mistake to mix imports from both ``pyspark.mllib`` and ``pyspark.ml`` in the same codebase?
- Q3. Why does model persistence matter for a real production deployment, rather than just retraining a model every time it's needed?

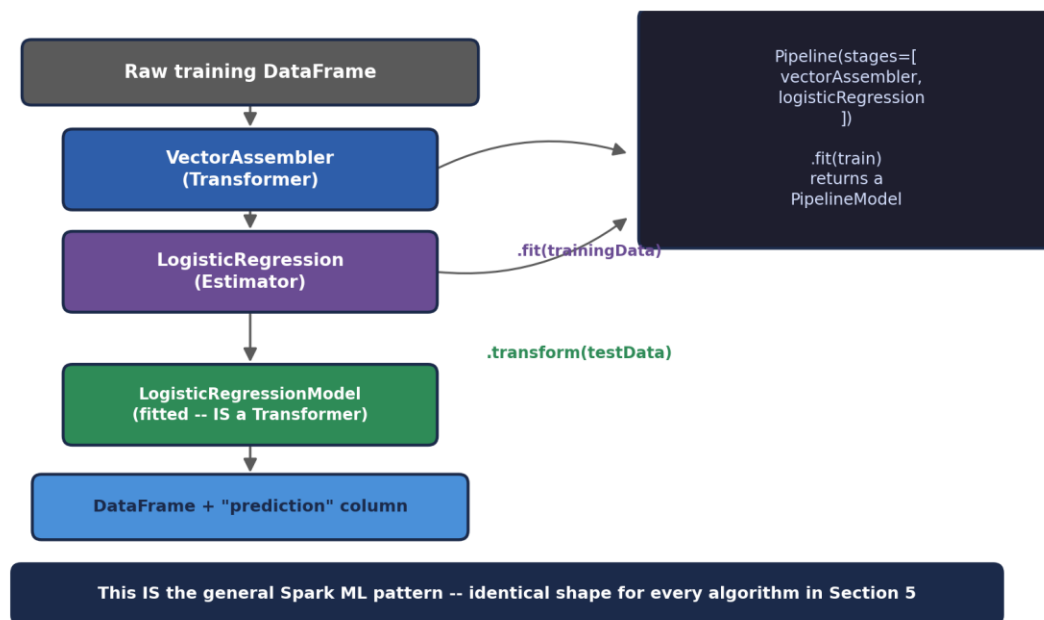
Hands-on Challenge:

Train any simple model on a small `DataFrame` (even from Section 1's exercise), save it using ``.write().overwrite().save(...)``, then load it back in the SAME script under a different variable name, and confirm the loaded model produces identical predictions to the original.

4. Key Concepts of Spark ML

This section formalizes the vocabulary Unit III's feature-extraction section introduced in passing: Transformer, Estimator, Pipeline, and Parameter -- the four building blocks every algorithm in Section 5 is built from.

The Spark ML Pipeline — Estimators, Transformers, `fit()`, `transform()`



The Spark ML Pipeline — Estimators, Transformers, `fit()`, `transform()`

4.1 DataFrame — the Universal Data Structure

Every Spark ML algorithm operates on a DataFrame (Unit IV), with data organized into named columns — commonly including a features column (a Vector, produced by VectorAssembler as in Unit III Section 10.4) and, for supervised algorithms, a label column holding the known answer.

4.2 Transformer

A Transformer converts one DataFrame into another by applying a `.transform()` method — it does NOT learn anything from data; it applies a fixed, already-determined rule. VectorAssembler (Unit III) and a fitted, trained model (like a LogisticRegressionModel) are both Transformers — the fitted model transforms a DataFrame of features into a DataFrame with an added prediction column.

4.3 Estimator

An Estimator learns from data via a `.fit()` method, and PRODUCES a Transformer as its output. LogisticRegression (before training) is an Estimator; calling `.fit(trainingData)` on it produces a LogisticRegressionModel, which is a

Transformer. This Estimator-produces-Transformer relationship is the single most important structural fact in all of Spark ML.

```
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(featuresCol="features", labelCol="label")
print(type(lr))          # LogisticRegression -- an ESTIMATOR (has .fit())

model = lr.fit(trainingData) # .fit() -- this is where actual training happens
print(type(model))         # LogisticRegressionModel -- a TRANSFORMER (has
                           # .transform())

predictions = model.transform(testData) # applying the now-fixed, trained rule
```

4.4 Pipeline

A Pipeline chains multiple Transformers and Estimators together into ONE combined Estimator — calling `.fit()` on the whole Pipeline runs every stage in sequence, and produces a single `PipelineModel` (itself a Transformer) that applies every stage's learned transformation together in one `.transform()` call. This is exactly Unit III Section 10.5's pattern, now including a trained predictive model as the final stage rather than just feature engineering.

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StringIndexer
from pyspark.ml.classification import LogisticRegression

indexer = StringIndexer(inputCol="category", outputCol="categoryIndex")
assembler = VectorAssembler(inputCols=["age", "income", "categoryIndex"],
                             outputCol="features")
lr = LogisticRegression(featuresCol="features", labelCol="label")

pipeline = Pipeline(stages=[indexer, assembler, lr])

pipelineModel = pipeline.fit(trainingData) # runs ALL three stages'
fit()/transform()
predictions = pipelineModel.transform(testData)
```

4.5 Parameter

Every Estimator and Transformer exposes PARAMETERS controlling its behavior — set either through constructor arguments or setter methods. Understanding a parameter's default and its effect is often the difference between a mediocre and a well-tuned model.

```
# Two equivalent ways to set parameters
```

```
lr = LogisticRegression(maxIter=50, regParam=0.1, featuresCol="features",
labelCol="label")

lr2 = LogisticRegression()
lr2.setMaxIter(50).setRegParam(0.1).setFeaturesCol("features").setLabelCol("label")

# Inspecting all available parameters and their current values
print(lr.explainParams())
```



An Assembly Line With Two Kinds of Stations

A Pipeline is a factory assembly line with two kinds of stations. A Transformer station is a machine already calibrated and bolted down -- raw material goes in one side, a fixed, unchanging modification comes out the other, every single time. An Estimator station is different -- it's an APPRENTICE machine that watches a batch of material go by, LEARNS the right calibration from what it observes (.fit()), and only THEN bolts itself down into a fixed Transformer station for all future material. A Pipeline's .fit() call is the one moment every apprentice station on the line gets to watch, learn, and permanently calibrate itself, in order, before the whole line is ready for real production.



Why the Pipeline abstraction matters beyond convenience

Bundling feature engineering and model training into a single Pipeline object isn't just less code to write — it eliminates an entire category of production bugs where a model is retrained with slightly different feature engineering than what was used to originally validate it (a mismatch commonly called training-serving skew). Persisting a fitted PipelineModel (Section 3.3) guarantees that every future prediction uses EXACTLY the same feature engineering the model was validated against.



Check yourself — Section 4

Q1. What's the precise relationship between an Estimator and a Transformer — which one PRODUCES the other?

Q2. Why does a fitted PipelineModel solve a real production problem (training-serving skew) beyond just code convenience?

Q3. Give one example each of a Transformer and an Estimator from this section's code, and explain why each is classified that way.

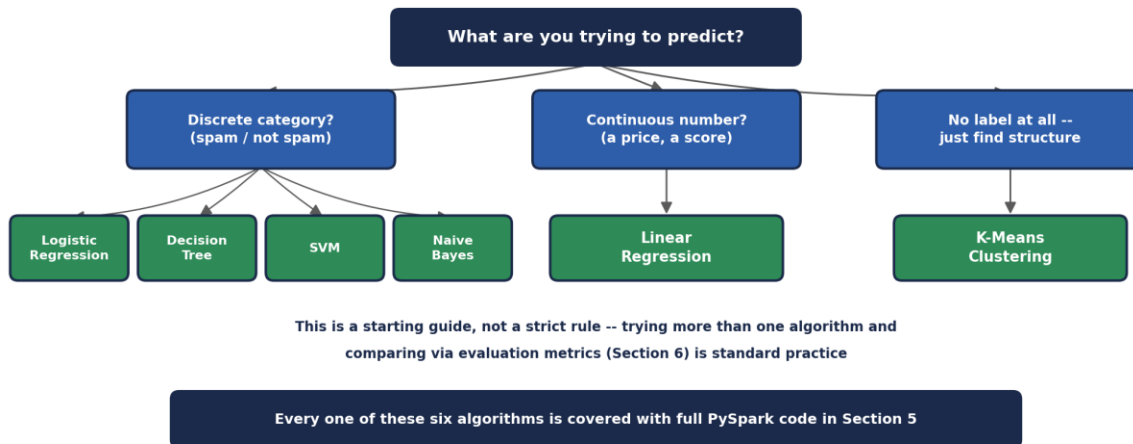
Hands-on Challenge:

Build a 3-stage Pipeline of your own (any combination of a StringIndexer, a VectorAssembler, and one algorithm from Section 3.2's imports), fit it on a small DataFrame, and call `.explainParams()` on the untrained algorithm stage before fitting to see its full list of configurable parameters.

5. Spark ML Algorithms Using PySpark

This is the practical core of the unit: six algorithms, each following the exact Estimator/Transformer pattern from Section 4, covering all three techniques from Section 2's taxonomy. Every algorithm below is trained, applied, and inspected using real, runnable PySpark code.

Choosing an Algorithm — A Starting Guide



Choosing an Algorithm — A Starting Guide



Six Different Specialists in the Same Hospital

Think of these six algorithms as six specialists working in the same hospital (Spark ML), all trained at the same medical school (the Estimator/Transformer pattern) and all using the same patient intake forms (a DataFrame with a features column) -- but each specialist reaches for a different diagnostic APPROACH depending on the case: a straight-line reasoner (Linear Regression) for continuous measurements, a probability-based diagnostician (Logistic Regression) for yes/no calls, a specialist who thinks in a branching series of questions (Decision Tree), a specialist who finds the widest possible separating boundary between cases (SVM), a specialist who reasons from base rates and independent symptoms (Naive Bayes), and a specialist who simply groups similar-looking patients together with no diagnosis at all (K-Means).

5.1 Linear Regression

Linear Regression predicts a CONTINUOUS numeric value by fitting the best straight-line (or hyperplane, for multiple features) relationship between the input features and the target. It's the foundational regression algorithm — often the first one tried on any regression problem precisely because it's simple, fast, and highly interpretable (each feature gets a coefficient showing its direction and strength of effect).

```

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression

# Example: predicting house price from size and number of bedrooms
data = spark.createDataFrame([
    (1200, 2, 250000.0), (1500, 3, 310000.0), (1800, 3, 365000.0),
    (2200, 4, 420000.0), (1000, 2, 210000.0), (2500, 4, 480000.0)
], ["size_sqft", "bedrooms", "price"])

assembler = VectorAssembler(inputCols=["size_sqft", "bedrooms"],
    outputCol="features")
assembled = assembler.transform(data)

train, test = assembled.randomSplit([0.8, 0.2], seed=42)

lr = LinearRegression(featuresCol="features", labelCol="price")
model = lr.fit(train)

print(f"Coefficients: {model.coefficients}") # effect of size_sqft, bedrooms
print(f"Intercept: {model.intercept}")

predictions = model.transform(test)
predictions.select("price", "prediction").show()

```

5.2 Logistic Regression

Despite the name (Section 2's naming trap), Logistic Regression is a CLASSIFICATION algorithm. It models the PROBABILITY that a given row belongs to a category, using a logistic (sigmoid) function to squeeze a linear combination of features into a 0-to-1 probability, then applies a threshold (commonly 0.5) to make the final categorical prediction.

```

from pyspark.ml.classification import LogisticRegression

# Example: predicting customer churn (1 = churned, 0 = retained)
data = spark.createDataFrame([
    (5, 2, 1), (24, 0, 0), (2, 5, 1), (36, 1, 0), (1, 4, 1), (48, 0, 0)
], ["tenure_months", "support_tickets", "churned"])

assembler = VectorAssembler(inputCols=["tenure_months", "support_tickets"],
    outputCol="features")
assembled = assembler.transform(data)
train, test = assembled.randomSplit([0.8, 0.2], seed=42)

logr = LogisticRegression(featuresCol="features", labelCol="churned")
model = logr.fit(train)

```

```
predictions = model.transform(test)
predictions.select("churned", "prediction", "probability").show(truncate=False)
# 'probability' is a vector: [P(class=0), P(class=1)]
```

5.3 Decision Tree

A Decision Tree classifies (or, in a regression variant, predicts a number) by learning a series of branching yes/no questions about the features — "is age > 30? if yes, is income > \$50k? ..." — ending in a leaf that holds the prediction. Its major appeal is interpretability: the learned tree can be read and understood directly, unlike many other algorithms' internal coefficients.

```
from pyspark.ml.classification import DecisionTreeClassifier

dt = DecisionTreeClassifier(featuresCol="features", labelCol="churned",
maxDepth=5)
model = dt.fit(train)

predictions = model.transform(test)
predictions.select("churned", "prediction").show()

# Decision Trees expose FEATURE IMPORTANCE -- which inputs mattered most
print(model.featureImportances)
# e.g. (2,[0,1],[0.72,0.28]) -- tenure_months mattered more than support_tickets
here

# The tree structure itself can be printed for direct interpretation
print(model.toDebugString)
```

A common Decision Tree pitfall: overfitting

A Decision Tree grown without a depth limit will keep splitting until it perfectly memorizes the training data, including its noise — producing excellent training accuracy but poor performance on new, unseen data (a classic OVERFITTING failure). The `maxDepth` parameter above is the most direct lever for controlling this trade-off, and tuning it against a held-out test set (Section 6) is standard practice.

5.4 K-Means Clustering

K-Means is Spark ML's clustering (unsupervised) algorithm. Given a target number of clusters K , it iteratively assigns each row to its nearest cluster CENTER, then recomputes each center as the average of its assigned rows, repeating until the assignments stabilize — conceptually similar in its iterative, converge-to-a-fixed-point structure to Unit III's Pregel supersteps, though solving a completely different problem.

```

from pyspark.ml.clustering import KMeans

# Example: segmenting customers by spending behavior -- NO labels at all
data = spark.createDataFrame([
    (12, 400), (15, 420), (60, 50), (65, 45), (11, 390), (58, 60)
], ["purchase_frequency", "avg_order_value"])

assembler = VectorAssembler(inputCols=["purchase_frequency", "avg_order_value"],
    outputCol="features")
assembled = assembler.transform(data)

kmeans = KMeans(featuresCol="features", k=2, seed=42) # k -- how many clusters
to find
model = kmeans.fit(assembled) # note: NO label column -- unsupervised

predictions = model.transform(assembled)
predictions.select("purchase_frequency", "avg_order_value", "prediction").show()
# 'prediction' here is a CLUSTER ID (0, 1, ...), not a class label

print(f"Cluster centers: {model.clusterCenters()}")

```

☑ Choosing K -- the Elbow Method

K-Means requires you to CHOOSE the number of clusters ahead of time, unlike the supervised algorithms above where the number of classes is simply given by the data. A common technique, the "elbow method," trains the model for several values of K, plots each K against its total within-cluster distance (available via `model.summary.trainingCost``), and looks for the point where adding more clusters stops meaningfully improving the fit.

5.5 Support Vector Machine (SVM)

SVM is a classification algorithm that finds the widest possible separating boundary (the "maximum margin") between classes, rather than just any boundary that happens to separate them. Spark ML's LinearSVC implements the linear variant, well-suited to high-dimensional data with a fairly clean separation between classes.

```

from pyspark.ml.classification import LinearSVC

svm = LinearSVC(featuresCol="features", labelCol="churned", maxIter=20,
    regParam=0.1)
model = svm.fit(train)

predictions = model.transform(test)
predictions.select("churned", "prediction").show()

print(f"Coefficients: {model.coefficients}")

```

```
print(f"Intercept: {model.intercept}")
```

A real limitation worth knowing

Spark ML's `LinearSVC` supports only BINARY classification (two classes) and only a LINEAR decision boundary — unlike some other ML libraries' SVM implementations, it does not natively support the "kernel trick" for non-linear boundaries or more than two classes directly (multi-class SVM requires a one-vs-rest wrapping strategy built on top). This is a genuine, sometimes surprising constraint when migrating SVM code from a single-machine library like scikit-learn.

5.6 Naive Bayes

Naive Bayes is a classification algorithm based directly on Bayes' theorem from probability theory, making the simplifying ("naive") assumption that all features are independent of each other given the class — an assumption that's rarely literally true, yet the algorithm performs surprisingly well in practice, especially for text classification (spam detection, sentiment analysis) where feature counts (word frequencies) are the natural input.

```
from pyspark.ml.classification import NaiveBayes

# Naive Bayes requires NON-NEGATIVE feature values (it models feature
# counts/frequencies)
nb = NaiveBayes(featuresCol="features", labelCol="churned", smoothing=1.0)
model = nb.fit(train)

predictions = model.transform(test)
predictions.select("churned", "prediction", "probability").show(truncate=False)

# The 'smoothing' parameter (Laplace smoothing) prevents a zero-probability
# feature value from making the whole prediction collapse to zero
```

Where Naive Bayes still earns its place in production

Despite far more sophisticated text-classification algorithms existing today, Naive Bayes remains a common first-pass spam filter and sentiment classifier baseline in real production systems, precisely because it trains extremely fast (a single pass over the data computing frequency counts, with no iterative optimization loop) and performs competitively on exactly the sparse, high-dimensional, word-count-style features text classification naturally produces (recall Unit III Section 10.2's HashingTF/IDF features as a natural input).

5.7 Comparing All Six Algorithms

Algorithm	Technique	Best Suited For
Linear Regression	Regression	Predicting a continuous value with a roughly linear relationship to features
Logistic Regression	Classification	Binary/multi-class classification with a fast, interpretable baseline
Decision Tree	Classification	Problems needing an interpretable, human-readable decision logic
K-Means	Clustering	Discovering natural groupings with no labeled data at all
SVM (LinearSVC)	Classification	High-dimensional, linearly (or near-linearly) separable binary classification
Naive Bayes	Classification	Fast text classification with sparse, count-based features

Check yourself — Section 5

Q1. Which of these six algorithms are classification, which are regression, and which is clustering — sort them into Section 2's taxonomy from memory.

Q2. Why can an unlimited-depth Decision Tree overfit its training data, and what parameter directly controls this risk?

Q3. Why does K-Means's training call to `.fit()` NOT require a label column, while every other algorithm in this section does?

Q4. Why does Naive Bayes remain a common production baseline for text classification despite more sophisticated alternatives existing?

Hands-on Challenge:

Using any small dataset with both a numeric target and a binary category column (or the examples above), train BOTH a Linear Regression model (predicting the numeric target) and a Logistic Regression model (predicting the category), printing each model's coefficients and comparing their learned relationships to what you'd intuitively expect from the data.

6. Evaluation and Performance Metrics

Training a model (Section 5) is only half the job — knowing whether it's actually GOOD requires rigorous evaluation. This closing section covers the confusion matrix and its derived metrics for classification, RMSE/MAE/R² for regression, and correlation heat maps for understanding feature relationships before modeling even begins.

6.1 The Confusion Matrix

For a binary classifier, every prediction falls into exactly one of four buckets, comparing what was PREDICTED against what was ACTUALLY true.

The Confusion Matrix & Derived Metrics

Actual vs. Predicted -- Binary Classification

	Predicted: YES	Predicted: NO
Actual: YES	TRUE POSITIVE (TP) predicted YES, actually YES	FALSE NEGATIVE (FN) predicted NO, actually YES
Actual: NO	FALSE POSITIVE (FP) predicted YES, actually NO	TRUE NEGATIVE (TN) predicted NO, actually NO

$$\text{Accuracy} = (TP + TN) / \text{Total}$$

$$\text{Precision} = TP / (TP + FP) \quad \text{-- of predicted YES, how many really are YES}$$

$$\text{Recall} = TP / (TP + FN) \quad \text{-- of actual YES, how many did we catch}$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The Confusion Matrix & Derived Metrics

```
from pyspark.mllib.evaluation import MulticlassMetrics

# MulticlassMetrics needs an RDD of (prediction, label) pairs -- one of the
# few places pyspark.mllib (Section 3.1) is still commonly used, specifically
# for its detailed metrics support
```

```
predictionAndLabels = predictions.select("prediction", "churned").rdd.map(tuple)
metrics = MulticlassMetrics(predictionAndLabels)

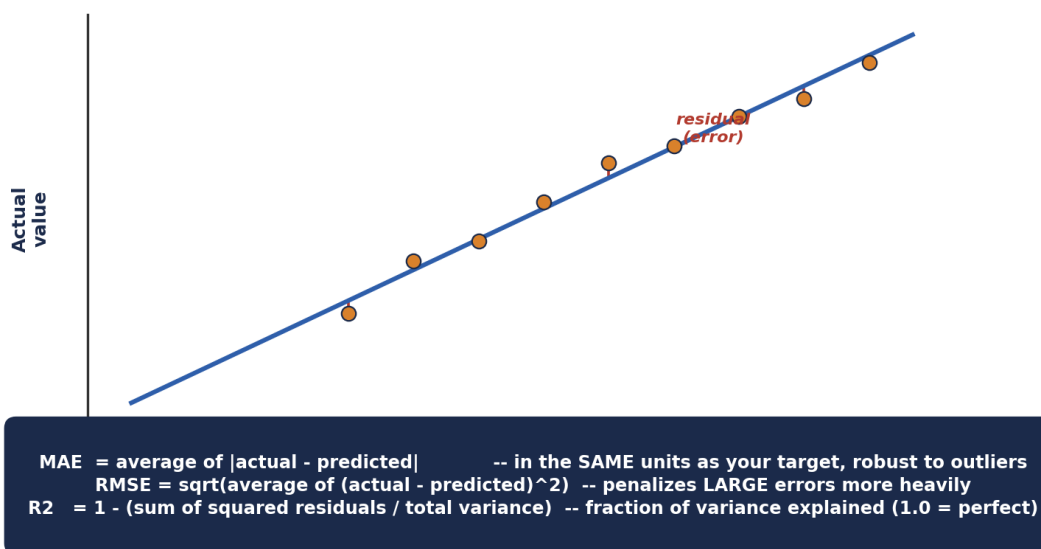
print(f"Confusion matrix:\n{metrics.confusionMatrix().toArray()}")
print(f"Accuracy: {metrics.accuracy}")
print(f"Precision (label=1): {metrics.precision(1.0)}")
print(f"Recall (label=1): {metrics.recall(1.0)}")
print(f"F1 Score (label=1): {metrics.fMeasure(1.0)}")
```

⚠ Why accuracy alone can be dangerously misleading

Imagine a fraud dataset where only 1% of transactions are actually fraudulent. A model that NAIVELY predicts "not fraud" for every single transaction achieves 99% accuracy while catching ZERO actual fraud — a completely useless model with an impressive-sounding number. This is precisely why precision, recall, and F1 (which specifically account for how the minority class is handled) matter far more than raw accuracy on imbalanced datasets, a genuinely common real-world situation across fraud detection, disease diagnosis, and rare-event prediction generally.

6.2 Regression Metrics — RMSE, MAE, and R^2

Regression Evaluation Metrics — RMSE, MAE, R^2



Regression Evaluation Metrics — RMSE, MAE, R^2


```

from pyspark.ml.evaluation import RegressionEvaluator

# Using the Linear Regression model and predictions from Section 5.1
evaluator = RegressionEvaluator(labelCol="price", predictionCol="prediction")

rmse = evaluator.setMetricName("rmse").evaluate(predictions)
mae = evaluator.setMetricName("mae").evaluate(predictions)
r2 = evaluator.setMetricName("r2").evaluate(predictions)

print(f"RMSE: {rmse:.2f}")    # same units as price -- typical prediction error,
                             # penalizes outliers
print(f"MAE: {mae:.2f}")    # same units as price -- typical prediction error,
                             # robust to outliers
print(f"R2: {r2:.4f}")      # 0 to 1 (can go negative for a very poor model) --
                             # variance explained

```

RMSE and MAE are both measured in the SAME units as the thing being predicted (dollars, for a house price model), making them directly interpretable — "our model is typically off by about \$15,000." The key difference: RMSE squares each error before averaging, so a few very large mistakes inflate RMSE much more than MAE, which treats every unit of error equally. R^2 instead answers a different question entirely: what FRACTION of the target's total variance does the model explain, independent of the target's units — useful for judging a model's relative quality across different datasets or targets.

6.3 Correlation Heat Maps — Understanding Features Before Modeling

Before (and after) training any model, it's standard practice to compute the CORRELATION between every pair of numeric features, visualized as a heat map — a grid where color intensity represents how strongly two features move together. This surfaces two things immediately: which features are likely predictive of the target, and which features are REDUNDANT with each other (highly correlated features add little extra information and can sometimes hurt certain models).

```

from pyspark.ml.stat import Correlation
from pyspark.ml.feature import VectorAssembler

numericCols = ["size_sqft", "bedrooms", "price"]
assembler = VectorAssembler(inputCols=numericCols, outputCol="corrFeatures")
vectorDF = assembler.transform(data).select("corrFeatures")

corrMatrix = Correlation.corr(vectorDF, "corrFeatures",
                              method="pearson").collect()[0][0]
print(corrMatrix.toArray())

# Converting to a pandas DataFrame (safe here -- a correlation matrix is always
# small, unlike the full dataset) to actually draw a heat map
import pandas as pd

```

```
import seaborn as sns
import matplotlib.pyplot as plt

corrPandas = pd.DataFrame(corrMatrix.toArray(), columns=numericCols,
index=numericCols)
sns.heatmap(corrPandas, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Feature Correlation Heat Map")
plt.show()
```

✓ Reading a correlation heat map correctly

A correlation value near +1 means two features increase together strongly; near -1 means one increases as the other decreases strongly; near 0 means little to no LINEAR relationship (though a strong NON-linear relationship could still exist and simply not show up as a high Pearson correlation). Always compute this BEFORE finalizing a feature set — discovering two features are 0.98 correlated after a model already trained poorly wastes time that a five-minute heat map check upfront would have saved.

📄 A Report Card vs. an X-Ray Before Surgery

RMSE, MAE, and the confusion matrix are a REPORT CARD, issued AFTER the model has already been trained and tested -- a final grade telling you how well the model actually performed. A correlation heat map is more like an X-ray taken BEFORE surgery even begins -- a look inside the data's own structure, revealing which features are likely to matter and which are simply duplicating information another feature already provides, informed decisions made before committing to the expensive process of actually training a model.

🏗️ How real teams actually use this evaluation toolkit together

A realistic model-development workflow uses a correlation heat map FIRST, during initial data exploration, to prune obviously redundant features and spot likely predictors; trains one or more candidate algorithms from Section 5; and evaluates EVERY candidate using the metrics from this section — confusion-matrix-derived precision/recall for classification tasks, RMSE/MAE/R² for regression tasks — before choosing which model, and which specific configuration of its parameters (Section 4.5), actually goes into production.

🧠 Check yourself — Section 6

Q1. Why can a model with 99% accuracy still be a genuinely bad, even dangerous, model in a fraud-detection context?

Q2. What's the practical difference between RMSE and MAE, and when would you specifically care about that difference?

Q3. Why is checking a correlation heat map BEFORE training a model considered good practice, rather than just evaluating after the fact?

Hands-on Challenge:

Using the Linear Regression example from Section 5.1, compute RMSE, MAE, and R^2 on the test set, then compute and print the full correlation matrix for all numeric columns in the training data using `Correlation.corr()`. Identify which feature appears most strongly correlated with the target.

Unit VI Review

Quick Review Quiz

Three questions covering the full breadth of this unit. Try to answer each from memory before checking against the sections above.

1. Q1. What's the precise relationship between an Estimator and a Transformer in Spark ML, and give one example of each?
2. Q2. Why is Logistic Regression classified as a classification algorithm despite its name, and name two other classification algorithms covered in this unit?
3. Q3. Why can a model with very high accuracy still be practically useless on an imbalanced dataset, and which metrics specifically address this?

☒ Answer key (cover before peeking)

A1: An Estimator **LEARNS** from data via `.fit()` and **PRODUCES** a Transformer as output; a Transformer applies an already-fixed rule via `.transform()` and learns nothing itself. Example: `LogisticRegression` is an Estimator; the `LogisticRegressionModel` it produces after `.fit()` is a Transformer.

A2: Logistic Regression predicts a discrete category (commonly binary) by modeling a probability and applying a threshold — it's classification, not regression, despite the historical name inherited from statistics. Other classification algorithms in this unit: Decision Tree, SVM (`LinearSVC`), and Naive Bayes.

A3: On an imbalanced dataset (e.g., 1% fraud), a model that always predicts the majority class achieves high accuracy while catching none of the minority class — precision, recall, and F1 (derived from the confusion matrix) specifically expose this failure by measuring performance on the minority class directly, rather than being dominated by the majority class's sheer volume.

Hands-On Exercise

Complete, end to end, using PySpark with a `SparkSession` available:

4. Create or load a `DataFrame` with at least 30 rows, including at least 3 numeric feature columns, one categorical column, and one binary label column (e.g., a simplified customer churn or loan-approval dataset).
5. Compute and inspect a correlation heat map (Section 6.3) across the numeric columns before doing anything else.
6. Build a Pipeline (Section 4.4) combining a `StringIndexer` for the categorical column, a `VectorAssembler`, and a Logistic Regression classifier, and fit it on an 80/20 train/test split.
7. Evaluate the fitted model's predictions using a confusion matrix and compute accuracy, precision, recall, and F1 (Section 6.1).

8. Separately, using just the numeric columns, train a Linear Regression model predicting one of the numeric columns from the others, and evaluate it with RMSE, MAE, and R^2 (Section 6.2).
9. Finally, train a K-Means model with $k=2$ or $k=3$ on the same numeric features (Section 5.4) and compare the resulting cluster assignments against your actual label column — do the unsupervised clusters roughly line up with the real labels?

 What this exercise is really testing

This single exercise deliberately exercises every syllabus topic in this unit at once: the ML workflow and PySpark basics (Section 1), correctly identifying classification vs. regression vs. clustering (Section 2), using the modern `pyspark.ml` API and Pipeline/Estimator/Transformer pattern (Sections 3-4), training multiple algorithm types (Section 5), and rigorously evaluating each one with the appropriate metric (Section 6) — a complete, realistic mini machine-learning project built entirely from this unit's material, and a fitting capstone across everything Units III through VI have built toward.